Exp : 5 Write the python program for Missionaries Cannibal problem.

Aim:

To write a python program for Missionaries Cannibal problem.

Algorithm:

Step 1: Create a data structure to represent the state of the problem, including the number of missionaries and cannibals on each side of the river and the position of the boat.

Step 2: Create a data structure to represent the state space of the problem, including initial state, the goal state, and the possible transitions between states.

Step 3: Use a search algorithm, such as depth-first search or breadth-first search, to explore the state space and find a solution.

Step 4: At each step of the search, generate all possible successor states from the current state by applying one of the allowed boat movements (two missionaries, two cannibals, or one of each).

Step 5: Check if the successor state is valid (i.e., no more cannibals than missionaries on either side of the river).

Step 6: If the successor state is valid, add it to the search tree and continue the search.

Step 7: If the successor state is the goal state, return the path from the initial state to the goal state.

Step 8: If there are no more possible successor states to explore, backtrack to the previous state and try another possible movement.

Program:

```
from collections import deque


def valid(ml, cl, mr, cr):
    if ml < 0 or cl < 0 or mr < 0 or cr < 0:
        return False
    if ml > 0 and ml < cl:
        return False
    if mr > 0 and mr < cr:
```

```
        return False
    return True

def bfs():
    moves = [(2,0), (0,2), (1,1), (1,0), (0,1)]
    start = (3,3,'L')
    goal = (0,0,'R')

    queue = deque([(start, [])])
    visited = set()

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path + [state]

        if state in visited:
            continue

        visited.add(state)

        ml, cl, boat = state

        for m, c in moves:
            if boat == 'L':
                new = (ml-m, cl-c, 'R')
            else:
                new = (ml+m, cl+c, 'L')
```

```
            nml, ncl, _ = new
            nmr = 3 - nml
            ncr = 3 - ncl

            if valid(nml, ncl, nmr, ncr):
                queue.append((new, path + [state]))

    return []

solution = bfs()

print("Solution:")
for state in solution:
    print(state)
```

Output:

```
========== RESTART: D
Solution:
(3, 3, 'L')
(3, 1, 'R')
(3, 2, 'L')
(3, 0, 'R')
(3, 1, 'L')
(1, 1, 'R')
(2, 2, 'L')
(0, 2, 'R')
(0, 3, 'L')
(0, 1, 'R')
(1, 1, 'L')
(0, 0, 'R')
```

Result:

The program was executed and the output was found to be correct.